SOP-005

Release and Deployment

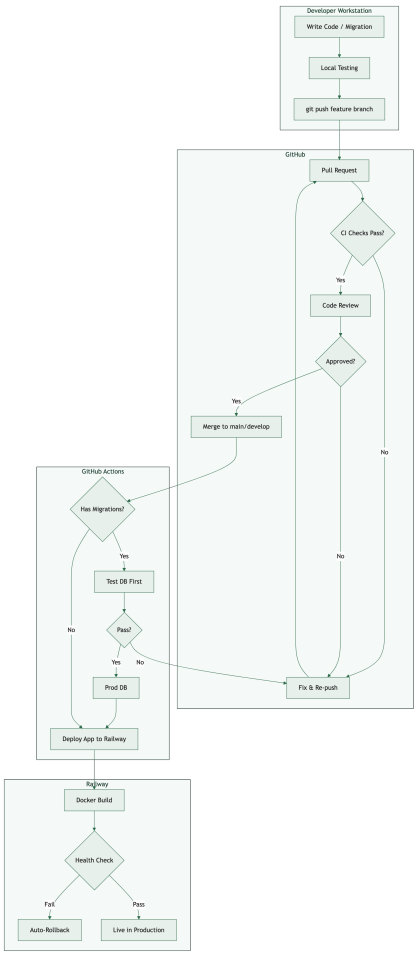
Effective: 2026-03-11 | **Owner:** Daniel Shanklin | **Applies to:** All Greenmark services deployed via Railway, Supabase, and GitHub Actions

BLUF

Every production change follows the same pipeline: **branch** → **test** → **review** → **merge** → **auto-deploy** → **verify** → **monitor.** Database migrations always deploy to the test project first. Rollbacks are service-specific — know your service's rollback path before you deploy. This SOP is the single source of truth for how code moves from a developer's machine to production.

1. Release Pipeline Overview

All Greenmark services follow a trunk-based deployment model. Code merges to a target branch (main or develop) trigger GitHub Actions, which deploy to Railway via the Railway CLI. Database migrations follow a separate, more cautious path through the Supabase test project before touching production.



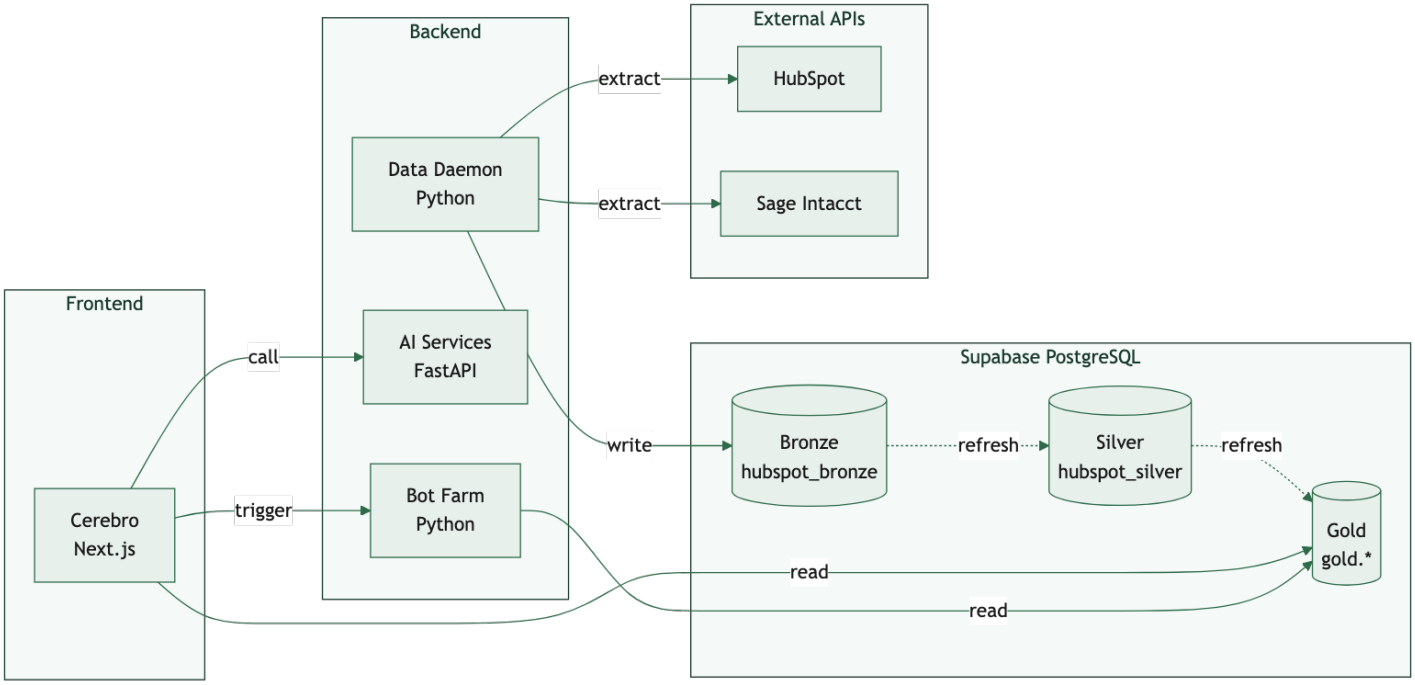
Stage	Action	Tools
1. Develop	Write code/migrations on feature branch	Git, IDE, Claude Code
2. Test Locally	Type checks, unit tests, lint	tsc, pytest, eslint
3. Push	Push feature branch, open PR	GitHub
4. CI Checks	Automated type check, unit tests, E2E tests	GitHub Actions

Stage	Action	Tools
5. Review	Code review, ADR compliance check	GitHub PR review
6. Merge	Merge to main/develop	GitHub
7. Deploy Migrations	Test DB first, then production	Supabase CLI
8. Deploy App	Railway CLI builds Docker, health check	GitHub Actions, Railway
9. Verify	Health endpoints, migration tracker, data checks	curl, SQL queries
10. Monitor	Logs, metrics, error rates for 5 min	Railway dashboard

2. Service Inventory

Greenmark operates five services, all deployed to Railway. Each has its own repository, deployment trigger, and health check.

Service	Language	Deploy Branch	Health Endpoint	Railway Env
Cerebro	Next.js / TS	main (prod), develop (dev)	`/api/health`	production, develop
Data Daemon	Python 3.11	develop	`/health`	develop
AI Services	Python 3.12 / FastAPI	main	`/ready`	production
Bot Farm	Python 3.12	main	(health check)	production
Portal	Node.js / Express	manual	`/health`	—



3. Tech Stack Reference

3.1 Infrastructure Components

Component	Role	Notes
GitHub github.com	Source control, CI/CD via Actions	All repos under greenmark-waste-solutions org
Railway railway.app	Container hosting, zero-downtime deploys	Docker-based, health check gating
Supabase supabase.com	PostgreSQL database, auth, RLS	Prod: wwmcgtyngnziepeynccz / Test: izmuckuepryqneebwwol

Component	Role	Notes
Docker docker.com	Containerization for all services	Multi-stage Alpine builds
Slack slack.com	Deployment failure notifications	Bot Farm alerts to channel C0AD03NG53Q

3.2 Build Tools

Tool	Version	Used By
Node.js	22	Cerebro, Portal
Python	3.11 – 3.12	Data Daemon, AI Services, Bot Farm
TypeScript	(project)	Cerebro
Supabase CLI	latest	Cerebro migrations
Railway CLI	latest	All service deployments

4. Pre-Flight Checklist

Before any merge to a deploy branch, verify every item. AI agents and humans alike must follow this sequence.

#	Check	How	Required For
1	**Type check passes**	`npx tsc --noEmit`	Cerebro
2	**Unit tests pass**	`pytest tests/ -v`	Daemon, Bot Farm
3	**E2E tests pass**	`pytest test_sales_e2e.py`	Bot Farm (main)
4	**Schema ownership verified**	See SOP-002	Any migration
5	**Migration tested on test DB**	`supabase db push --db-url`	Cerebro migrations
6	**IF NOT EXISTS used**	Manual review	All migrations
7	**GRANT statements included**	Manual review	New tables/views
8	**RLS policies included**	Manual review	Gold tables
9	**SECURITY DEFINER used**	Manual review	Forge functions
10	**No hard DELETEs**	`grep "DELETE FROM"`	All migrations
11	**Governance check passed**	check-governance skillflow	All changes
12	**ADRs consulted**	cerebro_adrs() search	Schema changes

> **AI agents: Items 4, 8, 9, and 11 are non-negotiable. Skipping governance or RLS has caused production security gaps before.**

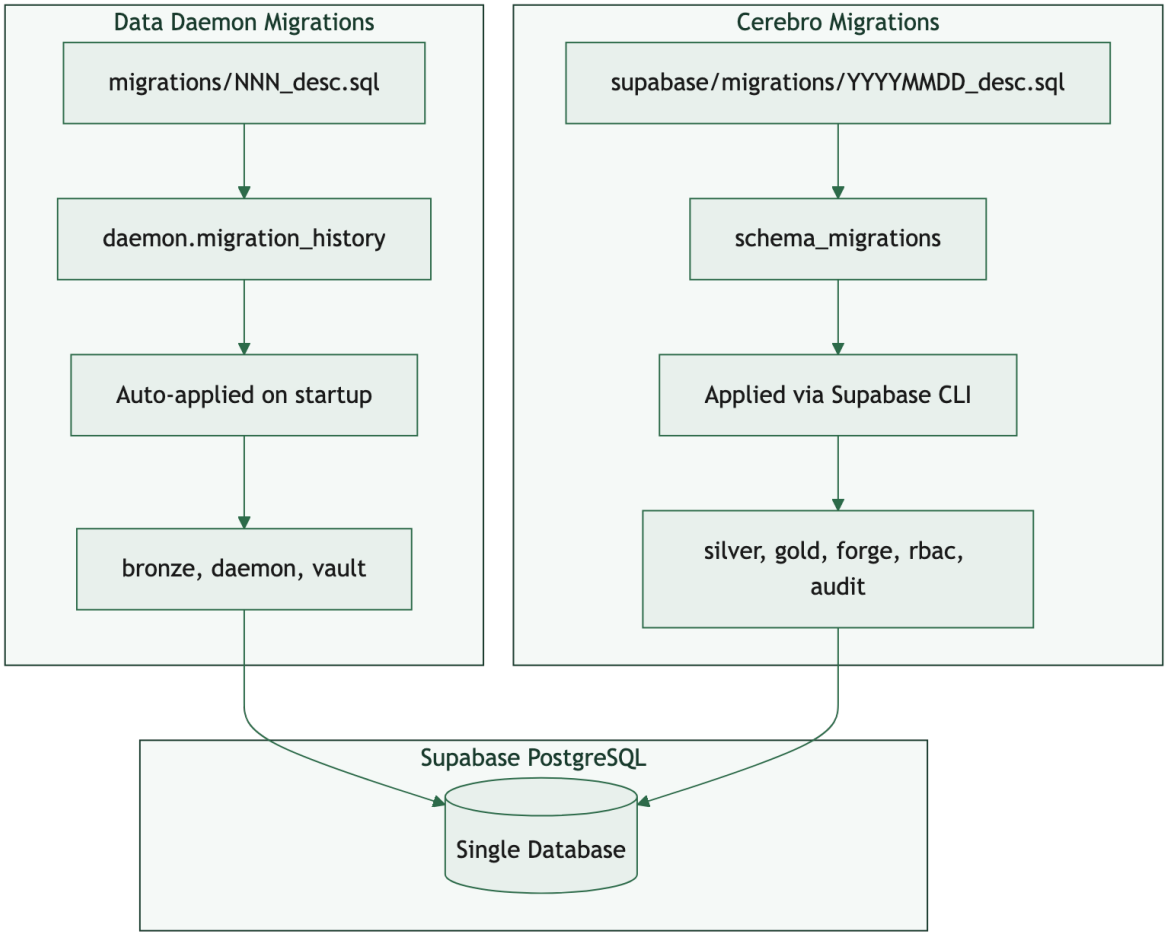
5. Database Migration Deployment

> **Migrations are the highest-risk part of any deployment. They modify shared state that cannot be easily undone. Follow this procedure exactly.**

5.1 Two Migration Systems, One Database

Greenmark uses two independent migration systems that write to the same Postgres database. Never cross the streams. See SOP-002 for full schema ownership table.

System	File Format	Tracker Table	Schemas Owned	Apply Method
Data Daemon	`NNN_desc.sql`	`daemon.migration_history`	bronze, daemon, vault	Auto on startup
Cerebro	`YYYYMMDDHHMMSS_desc.sql`	`supabase_migrations.schema_migrations`	silver, gold, forge, public, rbac, audit, platform	Supabase CLI



5.2 Cerebro Migration Procedure

Step	Action	Command / Detail
1	**Write migration**	`npx supabase migration new description_here`
2	**Test on test DB**	`npx supabase db push --db-url postgresql://postgres:<pw>@db.izmuckuepryqneebwwol.supabase.co:5432/postgres`
3	**Verify on test**	Check tables, RLS, functions exist. Run forge.refresh_all().
4	**Deploy to production**	`npx supabase db push --linked`
5	**Verify on production**	Same verification queries as step 3
6	**Populate gold tables**	`SELECT forge.refresh_all();`

5.3 Data Daemon Migration Procedure

Step	Action	Detail
1	**Write migration**	Place in data-daemon/migrations/ with next sequential number
2	**Test locally**	`DATABASE_URL=<test-url> python -m src.main`
3	**Verify**	Check daemon.migration_history for new entry
4	**Push to develop**	Railway redeploys, startup runs src/db/migrate.py
5	**Verify in prod**	Check daemon.migration_history in production

5.4 Migration Ordering Rules

When a release includes both silver and gold changes, apply in this order:

Order	What	Why
1	Bronze schema changes	Tables that silver reads from
2	Silver materialized views	Views that gold reads from
3	Gold tables	Tables with RLS
4	RLS policies	Security on gold tables
5	Forge refresh functions	Populate gold from silver
6	forge.refresh_all() update	Master refresh function
7	Initial population	SELECT forge.refresh_all()

> **Silver must exist before gold refresh functions that reference it. Gold tables must exist before RLS policies reference them. forge.refresh_all() must be updated before calling it.**

6. Application Deployment

Each service has a GitHub Actions workflow that deploys on merge. All use the Railway CLI.

6.1 Cerebro (Next.js)

Step	Action
1	Push to **main** (prod) or **develop** (dev)
2	GitHub Actions: checkout, Node 22, npm ci
3	Type check: npx tsc --noEmit
4	Install Railway CLI
5	railway up --service cerebro --environment <env> --detach
6	Railway builds Docker (multi-stage Alpine: deps → build → final)
7	Health check: /api/health (120s timeout, 3 retries)

Build args: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, NEXT_PUBLIC_ENV_LABEL

6.2 Data Daemon (Python)

Step	Action
1	Push to **develop**
2	GitHub Actions: checkout, install Railway CLI
3	railway up --service data-daemon --environment develop --detach
4	Startup: run migrations → init pool → start scheduler → start workers
5	Health check: /health (120s timeout, 5 retries)

6.3 AI Services (FastAPI)

Step	Action
1	Push to **main**
2	GitHub Actions: checkout, install Railway CLI
3	railway up --service cerebro-ai-services --environment production --detach
4	Health check: /ready (**300s timeout** — models take time to load)

> **AI Services has a persistent volume at /data/models that survives redeploys. If the model cache is corrupted, a fresh redeploy will re-download.**

6.4 Bot Farm (Python) — Most Sophisticated Pipeline

Step	Action	On Failure
1	Push to **main** or open PR	—
2	Setup Python 3.12, cache pip, install deps	—
3	Unit tests (all except E2E)	Slack alert
4	E2E tests against live Supabase (main push only)	Slack alert
5	railway up --service cerebro-bot-farm --environment production	Slack alert

7. Post-Deployment Verification

After every deployment, run through this verification matrix. Depth depends on what changed.

Changed	Verify	Expected
Application code	curl health endpoint → 200 OK	**200 OK**
Cerebro migrations	SELECT * FROM supabase_migrations.schema_migrations ORDER BY version DESC LIMIT 5	**New migration present**
Daemon migrations	SELECT * FROM daemon.migration_history ORDER BY applied_at DESC LIMIT 5	**New migration present**
Gold tables	SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname = 'gold'	**All rowsecurity = true**
Gold tables	SET request.jwt.claims to test entity; SELECT from gold table	**Only matching entity rows**
Forge functions	SELECT forge.refresh_all()	**No errors**
Silver views	SELECT COUNT(*) FROM hubspot_silver.<view>	**> 0**
RLS policies	SET ROLE service_role; SELECT from gold table	**Permission denied**

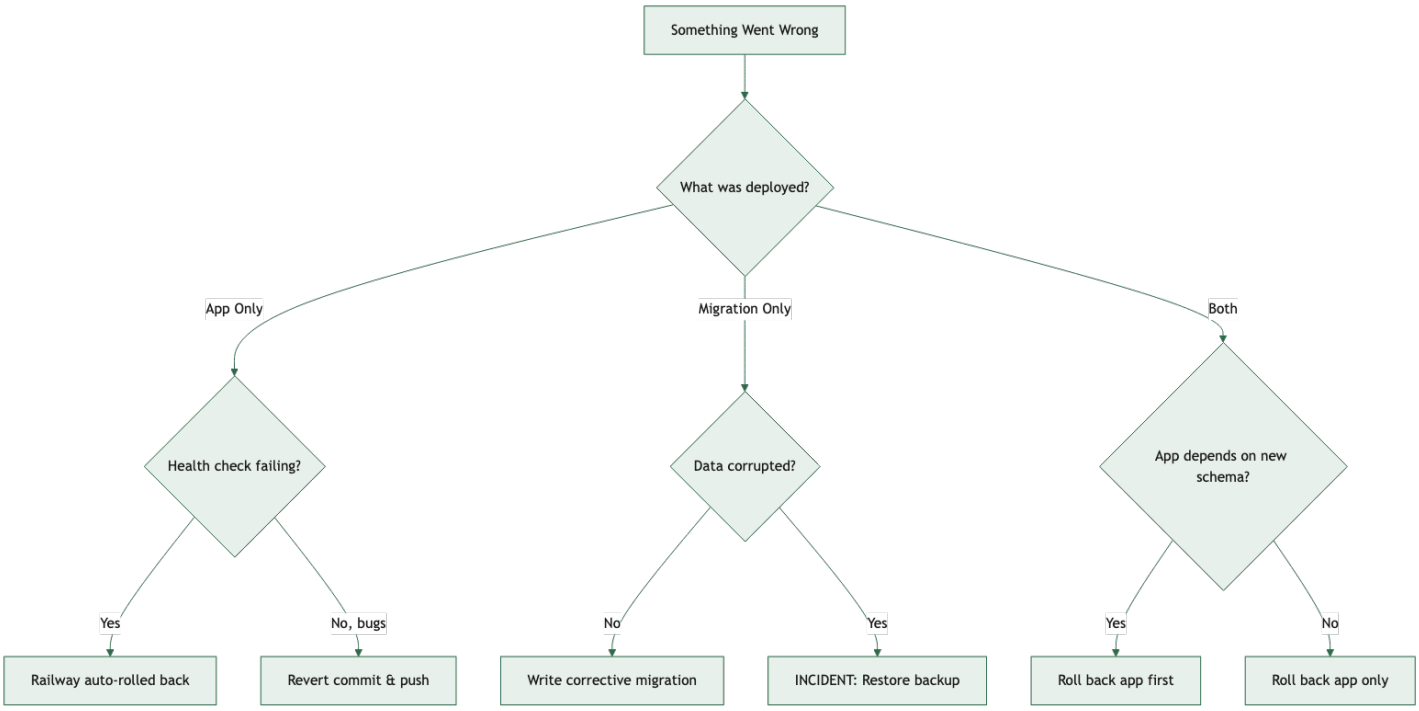
Smoke Test SQL

Run this after any deployment that touches the database:

```
1. SELECT version, name FROM supabase_migrations.schema_migrations ORDER BY version DESC LIMIT 3;
2. SELECT tablename FROM pg_tables WHERE schemaname = 'gold' AND rowsecurity = false; -- expect 0 rows
3. SELECT forge.refresh_all();
4. SELECT COUNT(*) FROM gold.pipeline_summary; -- expect > 0
5. SET request.jwt.claims = '{"entity_id":"ntx"}'; SELECT DISTINCT entity_id FROM gold.pipeline_summary; -- expect only 'ntx'
```

8. Rollback Procedures

Not all rollbacks are equal. The correct procedure depends on what was deployed and what went wrong.



8.1 Application Rollback

Method	When	How
Railway auto-rollback	Health check fails	Automatic — Railway reverts to previous deployment
Revert commit	Bugs in production	`git revert <sha> && git push origin main`
Redeploy previous	Need immediate rollback	Railway dashboard → previous deployment → Redeploy

8.2 Migration Rollback

> **Migrations cannot be automatically rolled back. There is no DOWN migration system. Write a corrective migration that undoes the damage, test on test DB, then apply.**

Scenario	Action
Added wrong column	`ALTER TABLE ... DROP COLUMN IF EXISTS bad_column;`
Created wrong table	`DROP TABLE IF EXISTS gold.bad_table;`
Dropped something needed	Recreate from original migration SQL
Data corrupted/lost	**INCIDENT:** Restore from Supabase backup (7-day daily retention)

8.3 Forge Function Rollback

Step	Action
1	**Stop the bleeding** — remove from forge.refresh_all()
2	**Fix the function** — CREATE OR REPLACE FUNCTION in corrective migration
3	**Re-run refresh** — SELECT forge.refresh_<table>()
4	**Verify** — check row counts and spot-check data

9. Environment Management

9.1 Environment Matrix

Environment	Purpose	Database	Railway	Branch
Production	Live users	`wwmcgtyngnziepeynccz`	production	main
Development	Pre-prod testing	`wwmcgtyngnziepeynccz (same!)`	develop	develop
Test	Migration testing only	`izmuckuepryqneebwwol`	—	—

> **Development and production share the same database. The Railway develop environment is for application testing (different URL), but database changes affect production immediately. This is why migrations MUST be tested on the test project first.**

9.2 Secret Management

Secret Type	Stored In	Accessed Via
API keys (HubSpot, Sage)	vault.secrets (encrypted)	secure.get_secret() in SQL
Railway tokens	GitHub Secrets	`${{ secrets.RAILWAY_TOKEN }}`
Supabase credentials	GitHub Secrets / Knox	CI/CD and local dev
Slack bot token	GitHub Secrets	Bot Farm failure alerts

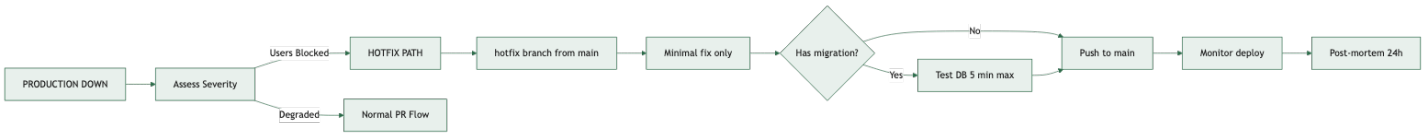
Rules: Never commit secrets to code. Rotate API keys quarterly. Railway tokens are per-environment. Vault access is audited.

9.3 Connection Strings

Target	Format
Prod (direct)	`postgresql://postgres.<ref>:<pw>@db.wwmcgtyngnziepeynccz.supabase.co:5432/postgres`
Prod (pooler)	`postgresql://postgres.<ref>:<pw>@aws-0-us-east-1.pooler.supabase.com:6543/postgres`
Test (direct)	`postgresql://postgres:<pw>@db.izmuckuepryqneebwwol.supabase.co:5432/postgres`

Note: Test uses direct (port 5432). Production should use Supavisor session-mode pooler (port 6543) for migrations per ADR-2026-03.

10. Emergency Hotfix Process



> **When production is broken and users are affected, use this expedited process. Minimal change only. Skip PR review ONLY if users are blocked.**

Step	Action	Notes
1	**Assess severity**	Users blocked = hotfix path. Degraded = normal PR flow.
2	**Create hotfix branch** from main	`git checkout -b hotfix/description main`
3	**Write minimal fix**	Fix the bug. Nothing else.
4	**Test migration (if any)**	Still required. 5 min max on test DB.
5	**Push to main**	Triggers auto-deploy
6	**Monitor deploy**	Watch health check, verify fix
7	**Post-mortem within 24h**	What broke, why, how to prevent
8	**Backport to develop**	Keep branches in sync

11. AI Agent Deployment Rules

When an AI agent (Claude Code, taskr-worker, etc.) is deploying changes:

Operation	AI Can Do Alone	Needs Human
Write migration SQL	**YES**	
Test migration on test DB	**YES**	
Create PR	**YES**	
Run type checks / tests	**YES**	
Merge PR		**REQUIRED**
Push to deploy branch		**REQUIRED**
Modify CI/CD pipelines		**REQUIRED**
Change secrets / env vars		**REQUIRED**
Database backups / restores		**REQUIRED**
Emergency hotfixes		**REQUIRED**

Mandatory Escalation Triggers

AI agents **must** escalate to a human when encountering:

Trigger	Why
Any RLS policy change	Security boundary — wrong policy = data leakage
Any GRANT/REVOKE statement	Permission changes affect all users
Any SECURITY DEFINER function	Runs with elevated privileges
Auth/authorization changes	Access control is business-critical
Railway config changes	Affects all deployments
Deploy failure after 2 retries	Indicates systemic issue, not transient
Conflicting ADR guidance	Human resolves architectural ambiguity

12. Monitoring and Observability

Signal	Where to Check	Red Flag
Health status	Railway dashboard	Any unhealthy service
Error logs	Railway logs	Error spike post-deploy
Migration status	migration_history / schema_migrations	Missing expected migration
Gold row counts	SELECT COUNT(*) FROM gold.*	Sudden drop to 0
RLS working	Entity isolation query	Cross-entity data leakage
API latency	Railway metrics	>2x increase post-deploy

13. Post-Mortem Template

When an incident occurs (data loss, extended downtime, security breach), document within 24 hours.

Section	Content
Title	Incident: [Short Title]
Metadata	Date, duration, severity (P1/P2/P3), services affected
Timeline	What happened, when detected, actions taken, resolution

Section	Content
Root Cause	What specifically broke and why
Impact	Users affected, data affected, duration
Resolution	What fixed it
Prevention	Specific action items with owners to prevent recurrence
Lessons Learned	What applies broadly across the organization

14. Release Cadence and Maturity

Greenmark uses continuous deployment — every merge to the deploy branch triggers production. There are no scheduled release windows or version numbers.

Practice	Status	Notes
Feature branches	**Active**	All work on feature branches
PR reviews	**Active**	Required before merge
Automated type checking	**Active**	Cerebro: tsc --noEmit
Automated unit tests	**Active**	Bot Farm
Automated E2E tests	**Active**	Bot Farm against live Supabase
Automated migration gate	**Planned**	GitHub Action → test DB on PR
Deploy notifications	**Partial**	Bot Farm only (Slack)
Git tags for releases	**Not active**	Consider for major releases
Changelog generation	**Not active**	Consider git log per release

15. GitHub Actions Workflow Structure

All deployment workflows live at *.github/workflows/deploy.yml* in each repo. They share a common structure but differ in CI depth. This section documents the YAML anatomy so anyone (human or AI) can read, modify, or create new workflows.

15.1 Workflow Anatomy

Every workflow has three sections: **trigger** (when), **environment** (what secrets), and **steps** (how). Here is the annotated structure:

```
name: Deploy <service>
on:
  push:
    branches: [main]     # or [develop], or [main, develop]
  pull_request:          # Bot Farm only — runs tests on PR
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      # === LANGUAGE SETUP (service-specific) ===
      - uses: actions/setup-node@v4  # Cerebro
        with: { node-version: 22 }
      - uses: actions/setup-python@v5 # Python services
        with: { python-version: '3.12' }
      # === CI CHECKS (service-specific) ===
      - run: npm ci && npx tsc --noEmit  # Cerebro
      - run: pip install -r requirements.txt # Python
      - run: pytest tests/ -v             # Bot Farm
      # === DEPLOY ===
      - run: npm i -g @railway/cli
      - run: railway up --service $SVC --environment $ENV --detach
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}
```

15.2 Workflow Comparison Matrix

Feature	Cerebro	Data Daemon	AI Services	Bot Farm
Trigger branch	main + develop	develop	main	main + PR
Language setup	Node 22	(none)	(none)	Python 3.12
Dependency install	npm ci	(none)	(none)	pip install
Type checking	tsc --noEmit	(none)	(none)	(none)
Unit tests	(none)	(none)	(none)	pytest
E2E tests	(none)	(none)	(none)	pytest E2E (main only)
pip caching	(none)	(none)	(none)	actions/cache
Multi-env deploy	Yes (prod/dev)	No (dev only)	No (prod only)	No (prod only)
Slack on failure	No	No	No	Yes
Docker health timeout	120s	120s	300s	(default)

15.3 Key GitHub Secrets

Secret Name	Used By	Purpose
`RAILWAY_TOKEN`	All services	Railway API auth for deploy
`RAILWAY_TOKEN_DEV`	Cerebro	Separate token for develop environment
`RAILWAY_SERVICE`	Data Daemon	Service name override
`RAILWAY_ENVIRONMENT`	Data Daemon	Environment name override
`SUPABASE_URL`	Bot Farm	E2E test database URL

Secret Name	Used By	Purpose
`SUPABASE_SERVICE_ROLE_KEY`	Bot Farm	E2E test auth
`SLACK_BOT_TOKEN`	Bot Farm	Failure notification

15.4 Adding a New Workflow

Step	Action
1	Copy the closest existing workflow (Bot Farm is the most complete)
2	Update: trigger branches, service name, environment name
3	Add language setup if needed (Node, Python, etc.)
4	Add CI checks appropriate to the service (tests, type check, lint)
5	Add RAILWAY_TOKEN to GitHub repo secrets
6	Test with a PR push first (if your workflow triggers on PR)
7	Add Slack notification for failures (copy from Bot Farm)

16. CI/CD Maturity Assessment

An honest comparison of Greenmark's release pipeline against industry benchmarks. Graded against two baselines: top-50 Silicon Valley firms circa 2024 and the same firms today (2026).

16.1 Current Grade: B-

> **Greenmark's CI/CD is above average for a 5-person team and ahead of most SMBs, but below the bar set by well-funded SV startups. The biggest gap is not tooling — it's coverage. Only 1 of 5 services has full test + deploy + alert.**

Dimension	Greenmark	SV Top 50 (2024)	SV Top 50 (2026)	Gap
Source Control	**A**	A	A	None — feature branches, PRs, protected main
CI: Type Safety	**A**	A	A	None — tsc --noEmit on every push (Cerebro)
CI: Unit Tests	**C+**	A	A	Only Bot Farm has unit tests in CI. Cerebro, Daemon, AI: none.
CI: E2E Tests	**B-**	A-	A	Bot Farm has live-DB E2E. Others have zero.
CI: Migration Gate	**D**	B+	A	Manual only. No CI runs migrations against test DB on PR.
CD: Auto-Deploy	**A**	A	A	None — merge-to-deploy on all active services
CD: Health Checks	**A**	A	A	None — Railway health checks + auto-rollback
CD: Canary/Blue-Green	**F**	B+	A	No staged rollouts. All-or-nothing deploys.
Observability	**C**	A-	A	Prometheus metrics (daemon), but no centralized logging, APM, or tracing.
Alerting	**D+**	A-	A	Slack alerts on Bot Farm only. No PagerDuty, no on-call rotation.
Rollback	**B**	A-	A	Railway auto-rollback for apps. No migration rollback system.
Secrets Management	**A-**	A	A	vault.secrets + GitHub Secrets + Knox. Well-separated.
Documentation	**A**	B	B+	Ahead of most. ADRs, SOPs, CLAUDE.md — unusually thorough.
AI-Agent Governance	**A**	N/A	B-	Pioneering. Most SV firms still lack AI deployment guardrails.

16.2 Strengths (What Top 50 Firms Would Envy)

Strength	Why It Matters
ADR + SOP discipline	Most SV firms have tribal knowledge. Greenmark has 35 ADRs and 5 SOPs with branded PDFs. This is rare even at large companies.
AI agent governance	Explicit rules for what AI can deploy alone vs. what needs human approval. Most firms haven't even thought about this yet.
Database security model	RLS + SECURITY DEFINER + service_role revocation + entity isolation. This is enterprise-grade multi-tenant security on a startup budget.
Merge-to-deploy simplicity	No release trains, no deploy queues, no approval chains. Fast feedback loop.

16.3 Gaps (Highest-Impact Improvements)

Priority	Gap	Fix	Effort
P0	No automated migration gate	GitHub Action: supabase db push --db-url on PR	S (2h)
P0	Deploy alerts on 1/5 services	Add Slack step to all deploy.yml workflows	S (1h)
P1	No unit tests for Cerebro or Daemon	Add pytest/jest to CI; start with critical paths	M (1-2 weeks)
P1	No centralized logging	Railway Logdrain → Axiom/Datadog/Loki	M (1 week)
P2	No canary deploys	Railway environments for canary; traffic split	L (2-4 weeks)

Priority	Gap	Fix	Effort
P2	No structured on-call	PagerDuty or Opsgenie + runbooks	M (1 week)

> **The two P0 items would move the grade from B- to B+ in about 3 hours of work. The P1 items get to A- over 2-3 weeks. Canary deploys and on-call are where the top 50 firms live — those are stretch goals for a team this size.**

Why This Exists

Greenmark's infrastructure grew organically — five services, two migration systems, one shared database, and AI agents writing code. Without a documented release process, each deployment was an oral tradition passed between sessions. This SOP codifies the pipeline so that any developer, AI agent, or on-call responder can deploy safely and roll back confidently.

The catalyst: a gold layer expansion requiring coordinated migrations across two systems (data-daemon bronze + cerebro silver/gold), where convention mismatches led to 7 discovered conflicts. A documented release process would have caught these earlier.

References

Document	Location
SOP-002: Database Migrations	`greenmark-docs/sops/SOP-002-database-migrations.md`
SOP-004: Technical Debt Remediation	`greenmark-docs/sops/SOP-004-technical-debt-remediation.md`
ADR-2026-03: Supabase Connection Strategy	`greenmark-docs/adrs/ADR-2026-03.md`
ADR-2026-04: Gold Tables Not Views	`greenmark-docs/adrs/ADR-2026-04.md`
ADR-2026-09: service_role Revoked	`greenmark-docs/adrs/ADR-2026-09.md`
ADR-2026-10: SECURITY DEFINER Refresh	`greenmark-docs/adrs/ADR-2026-10.md`
ADR-2026-11: Default Deny RLS	`greenmark-docs/adrs/ADR-2026-11.md`